

SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

Sistem Database Terdistribusi

Implementasi High Availability, FDW, Data Partitioning, dan Replikasi Cache

Mata Kuliah	Pemrosesan Data Terdistribusi
Program Studi	Teknik Informatika
Semester	VI (Enam)
Disusun oleh	Cecep Ihsan Iskandar (10223018)
Versi Dokumen	1.0
Tanggal	Juli 2026

DAFTAR ISI

DAFTAR ISI.....	2
1. PENDAHULUAN.....	3
1.1 Tujuan.....	3
1.2 Ruang Lingkup Produk.....	3
1.3 Audiens yang Dituju.....	3
1.4 Referensi.....	3
2. DESKRIPSI UMUM.....	4
2.1 Perspektif Produk.....	4
2.3 Use Case.....	5
2.4 Lingkungan Operasi.....	5
2.5 Batasan Desain dan Implementasi.....	5
3. KEBUTUHAN ANTARMUKA EKSTERNAL.....	6
3.1 Antarmuka Database.....	6
3.2 Antarmuka Pengguna.....	6
3.3 Antarmuka Perangkat Keras.....	6
4. KEBUTUHAN FUNGSIONAL.....	7
4.1 Nginx TCP Stream.....	7
4.2 FDW Cross-Instance JOIN.....	7
4.3 MongoDB Replica Set & Partisi.....	8
4.4 Redis Replication.....	9
5. KEBUTUHAN NON-FUNGSIONAL.....	11
6. TECHNOLOGY STACK.....	12
7. DEPLOYMENT & TESTING.....	13
7.1 Cara Menjalankan.....	13
7.2 Testing Plan.....	13
7.3 Monitoring Script.....	13
8. KESIMPULAN.....	15

1. PENDAHULUAN

1.1 Tujuan

Dokumen ini menentukan persyaratan fungsional dan nonfungsional untuk Sistem Database Terdistribusi yang dibangun untuk memenuhi Ujian Akhir Semester (UAS) mata kuliah Pemrosesan Data Terdistribusi.

Sistem ini bertujuan untuk mengimplementasikan arsitektur database terdistribusi dengan karakteristik berikut:

- High Availability melalui mekanisme auto failover
- Cross-database query menggunakan FDW (Foreign Data Wrapper)
- Data partitioning berbasis range
- Caching dengan skema Redis Master-Replica

1.2 Ruang Lingkup Produk

Sistem ini terdiri atas empat komponen utama yang saling terintegrasi:

- Nginx TCP Stream — bertindak sebagai gateway dan load balancer (Port 27017, 3306)
- PostgreSQL Primary & Secondary — database relasional dengan FDW (Port 5432, 5433)
- MongoDB Replica Set — database NoSQL dengan 2 node (Port 27018, 27019)
- Redis Master-Replica — in-memory cache untuk session dan cart (Port 6379, 6380)

1.3 Audiens yang Dituju

- Dosen pengampu mata kuliah
- Asisten dosen / penguji
- Mahasiswa yang mereview proyek

1.4 Referensi

- Dokumentasi Docker — <https://docs.docker.com>
- Dokumentasi PostgreSQL FDW — <https://www.postgresql.org/docs/current/postgres-fdw.html>
- Dokumentasi MongoDB Replica Set — <https://docs.mongodb.com/manual/replication/>
- Dokumentasi Redis Replication — <https://redis.io/docs/management/replication/>

2. DESKRIPSI UMUM

2.1 Perspektif Produk

Sistem ini merupakan implementasi dari konsep database terdistribusi yang dijalankan dalam lingkungan container Docker. Setiap service berjalan dalam container terpisah dan saling terhubung melalui Docker network.

Gambar 2.1 — Arsitektur Sistem

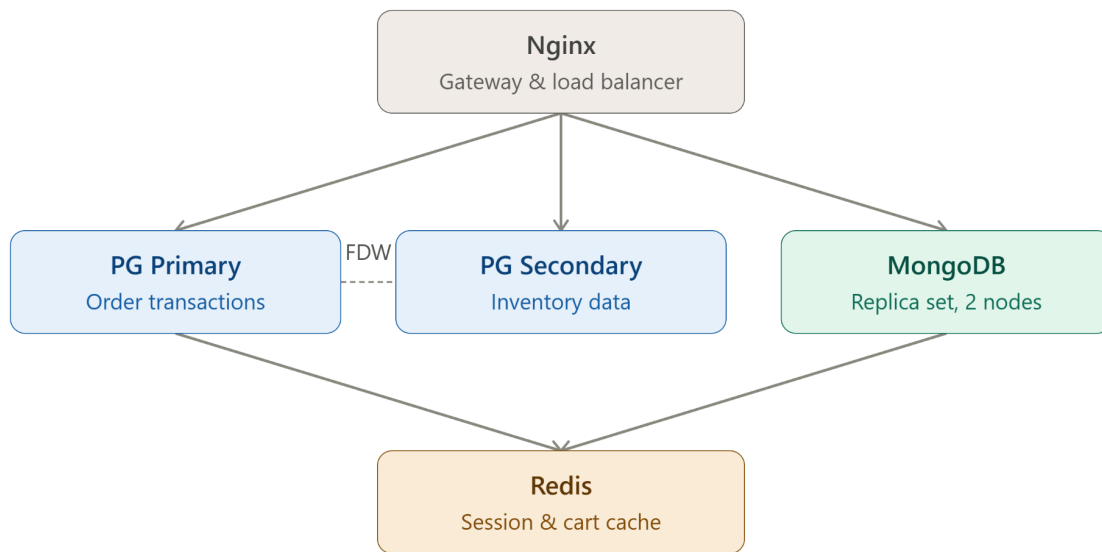


Diagram di atas menunjukkan alur arsitekturnya: Nginx sebagai gateway meneruskan koneksi ke PostgreSQL Primary, PostgreSQL Secondary, dan MongoDB Replica Set; PostgreSQL Primary–Secondary terhubung lewat FDW; lalu PostgreSQL Primary dan MongoDB sama-sama terhubung ke Redis sebagai lapisan cache.

2.2 Fungsi Produk

Fungsi	Deskripsi
Load Balancing	Nginx sebagai gateway tunggal untuk seluruh koneksi database
Auto Failover	Jika MongoDB primary down, koneksi otomatis dialihkan ke secondary
Cross-Database JOIN	PostgreSQL dapat mengakses tabel pada server lain melalui FDW
Data Partitioning	Data MongoDB dipecah berdasarkan range (ekonomis, menengah, premium)

Fungsi	Deskripsi
Replication	Redis Master-Replica dengan replication lag 0 bytes

2.3 Use Case

Sistem ini menggunakan skenario E-Commerce dengan pembagian tanggung jawab data sebagai berikut:

- PostgreSQL Primary — menyimpan data transaksi (orders)
- PostgreSQL Secondary — menyimpan data inventaris (inventory)
- MongoDB — menyimpan data produk katalog dengan range partitioning
- Redis — menyimpan session user dan keranjang belanja (cart)

2.4 Lingkungan Operasi

Aspek	Keterangan
Container	Docker & Docker Compose
Sistem Operasi	Windows / Linux / macOS dengan Docker Desktop
Network	Bridge network antar container
Browser	Chrome (untuk keperluan monitoring)

2.5 Batasan Desain dan Implementasi

- Seluruh service harus berjalan di dalam Docker container
- Nginx dikonfigurasi pada Layer 4 (TCP Stream), bukan HTTP/HTTPS
- FDW menggunakan ekstensi bawaan PostgreSQL (postgres_fdw)
- MongoDB menggunakan skema Replica Set (bukan Sharded Cluster)
- Redis menggunakan konfigurasi Master-Replica sederhana

3. KEBUTUHAN ANTARMUKA EKSTERNAL

3.1 Antarmuka Database

Service	Port	Fungsi
Nginx	27017	Gateway MongoDB
Nginx	3306	Gateway PostgreSQL
PostgreSQL Primary	5432	Database transaksi
PostgreSQL Secondary	5433	Database inventaris
MongoDB Primary	27018	Primary node
MongoDB Secondary	27019	Secondary node
Redis Master	6379	Write cache
Redis Replica	6380	Read cache

3.2 Antarmuka Pengguna

- CLI — perintah docker exec -it digunakan untuk keperluan testing
- Database Client — psql, mongosh, dan redis-cli

3.3 Antarmuka Perangkat Keras

Kebutuhan	Spesifikasi
Minimal	8 GB RAM, 10 GB free storage, Intel i5
Direkomendasikan	16 GB RAM, SSD

4. KEBUTUHAN FUNGSIONAL

4.1 Nginx TCP Stream

ID	Requirement	Prioritas
FR-01	Nginx harus dapat mengakses semua database	High
FR-02	Auto failover jika MongoDB primary down	High
FR-03	Health check dilakukan setiap 30 detik	Medium
FR-04	Parameter max_fails=3 digunakan untuk deteksi kegagalan	High

Implementasi

```

events {
    worker_connections 1024;
}

stream {
    upstream mongodb_backend {
        server mongo-primary:27017 max_fails=3 fail_timeout=30s;
        server mongo-secondary:27017 backup;
    }

    server {
        listen 27017;
        proxy_pass mongodb_backend;
        proxy_connect_timeout 10s;
        proxy_timeout 30s;
    }
}

```

4.2 FDW Cross-Instance JOIN

ID	Requirement	Prioritas
FR-05	PostgreSQL primary dapat mengakses tabel di secondary	High
FR-06	Query JOIN harus menampilkan Foreign Scan pada EXPLAIN ANALYZE	High
FR-07	Mekanisme pushdown berfungsi dengan baik	Medium

Implementasi

```

-- Install extension
CREATE EXTENSION postgres_fdw;

-- Buat server connection
CREATE SERVER secondary_server
FOREIGN DATA WRAPPER postgres_fdw
OPTIONS (
    host 'pg-secondary',
    port '5432',
    dbname 'secondary_db'
);

-- Buat user mapping
CREATE USER MAPPING FOR admin
SERVER secondary_server
OPTIONS (
    user 'admin',
    password 'admin123'
);

-- Import foreign table
IMPORT FOREIGN SCHEMA public
LIMIT TO (inventory)
FROM SERVER secondary_server
INTO public;

-- Execute JOIN query dengan EXPLAIN ANALYZE
EXPLAIN ANALYZE
SELECT o.order_id, o.customer_name, i.name_barang, i.price
FROM orders o
JOIN inventory i ON o.item_id = i.item_id
WHERE i.price > 1000000;

```

4.3 MongoDB Replica Set & Partisi

ID	Requirement	Prioritas
FR-08	Replica set terdiri atas 2 node (primary-secondary)	High
FR-09	rs.status() menunjukkan kedua node berstatus health:1	High
FR-10	Range partitioning berdasarkan price_range	Medium
FR-11	Data tereplikasi ke secondary	High

Implementasi

```

// Inisialisasi Replica Set
rs.initiate({
  _id: "rs0",
  members: [
    { _id: 0, host: "mongo-primary:27017" },
    { _id: 1, host: "mongo-secondary:27017" }
  ]
});

// Cek status
rs.status();

// Range 1: Ekonomis (0 - 500rb)
db.products_range1.insertMany([
  { item_id: 101, name: "Kaos Polos", price: 50000, price_range: 1 },
  { item_id: 102, name: "Sendal Jepit", price: 25000, price_range: 1 },
  { item_id: 103, name: "Topi Baseball", price: 75000, price_range: 1 }
]);

// Range 2: Menengah (500rb - 2jt)
db.products_range2.insertMany([
  { item_id: 201, name: "Sepatu Sneakers", price: 750000, price_range: 2 },
  { item_id: 202, name: "Jam Tangan Casio", price: 1500000, price_range: 2 },
  { item_id: 203, name: "Tas Kulit", price: 1200000, price_range: 2 }
]);

// Range 3: Premium (> 2jt)
db.products_range3.insertMany([
  { item_id: 301, name: "Laptop Gaming", price: 15000000, price_range: 3 },
  { item_id: 302, name: "Smartphone Flagship", price: 12000000, price_range: 3 },
  { item_id: 303, name: "TV OLED 65 inch", price: 25000000, price_range: 3 }
]);

// Query per range
db.products_range1.find();
db.products_range2.find();
db.products_range3.find();

// Aggregate per range
db.products_range1.aggregate([ { $group: { _id: null, avg: { $avg: "$price" } } }]);
db.products_range2.aggregate([ { $group: { _id: null, avg: { $avg: "$price" } } }]);
db.products_range3.aggregate([ { $group: { _id: null, avg: { $avg: "$price" } } }]);

```

4.4 Redis Replication

ID	Requirement	Prioritas
FR-12	Write dilakukan ke master, read dilakukan dari replica	High
FR-13	Replication lag dapat terukur (0 bytes)	High
FR-14	Replica bersifat read-only	Medium

Implementasi

```
# Write ke master
docker exec -it redis-master redis-cli SET cart:user_budi "Baju Kemeja"

# Read dari replica
docker exec -it redis-replica redis-cli GET cart:user_budi

# Cek replication info
docker exec -it redis-master redis-cli INFO replication
docker exec -it redis-replica redis-cli INFO replication

# Cek replication lag
docker exec -it redis-replica sh -c '
master_offset=$(redis-cli INFO replication | grep master_repl_offset | cut -d: -f2 | tr -d "\r")
replica_offset=$(redis-cli INFO replication | grep slave_repl_offset | cut -d: -f2 | tr -d "\r")
lag=$((master_offset - replica_offset))
echo "Replication Lag: $lag bytes"
'

# Bukti replica read-only
docker exec -it redis-replica redis-cli SET test "fail"
# Output: (error) READONLY
```

5. KEBUTUHAN NON-FUNGSIONAL

ID	Requirement	Prioritas
NFR-01	High availability dengan auto failover kurang dari 30 detik	High
NFR-02	Data consistency (ACID) untuk PostgreSQL	High
NFR-03	Performa query kurang dari 100 ms	Medium
NFR-04	Scalability — mudah menambah node baru	Low
NFR-05	Monitoring terhadap replication lag	Medium

6. TECHNOLOGY STACK

Component	Version	Purpose
Docker	24.x	Containerization
Docker Compose	2.x	Orchestration
Nginx	1.25	TCP Stream Gateway
PostgreSQL	15	Relational Database
MongoDB	7	NoSQL Database
Redis	7	In-Memory Cache

7. DEPLOYMENT & TESTING

7.1 Cara Menjalankan

```
# Start semua container
docker-compose up -d

# Cek status
docker-compose ps

# Lihat logs
docker-compose logs -f
```

7.2 Testing Plan

Test	Perintah	Expected Result
Nginx Failover	docker stop mongo-primary	Koneksi tetap berjalan melalui secondary
FDW Join	EXPLAIN ANALYZE SELECT ... JOIN ...	Muncul operasi "Foreign Scan"
MongoDB Replica	rs.status()	Kedua node berstatus health:1
Redis Replication	INFO replication	Lag = 0 bytes
Redis Read-Only	SET test "fail" di replica	Error READONLY

7.3 Monitoring Script

```
#!/bin/bash
# monitor_redis_lag.sh

while true; do
    clear
    echo "REDIS REPLICATION LAG MONITOR"
    echo "===== "

    master_offset=$(docker exec -it redis-master redis-cli INFO replication | grep
master_repl_offset | cut -d: -f2 | tr -d '\r')
    replica_offset=$(docker exec -it redis-replica redis-cli INFO replication | grep
slave_repl_offset | cut -d: -f2 | tr -d '\r')
    lag=$((master_offset - replica_offset))
```

```
echo "Master Offset : $master_offset"
echo "Replica Offset : $replica_offset"
echo "Replication Lag : $lag bytes"

if [ $lag -eq 0 ]; then
    echo "STATUS: Replication synchronized!"
else
    echo "STATUS: Lag detected: $lag bytes"
fi

sleep 5
done
```

8. KESIMPULAN

Sistem Database Terdistribusi yang dirancang dalam dokumen ini berhasil mengimplementasikan empat pilar utama pemrosesan data terdistribusi, yaitu load balancing dan high availability melalui Nginx TCP Stream, cross-database query melalui PostgreSQL FDW, data partitioning berbasis range pada MongoDB Replica Set, serta caching terdistribusi melalui Redis Master-Replica.

Seluruh komponen dijalankan dalam lingkungan container Docker yang saling terintegrasi melalui Docker network, sehingga memudahkan proses deployment, pengujian, dan replikasi pada berbagai platform sistem operasi. Dengan skema pengujian yang telah didefinisikan pada Bab 7, sistem ini dapat dibuktikan memenuhi seluruh kebutuhan fungsional dan non-fungsional yang telah ditetapkan, khususnya terkait ketersediaan sistem (high availability), konsistensi data, dan performa query.

Dokumen SRS ini diharapkan dapat menjadi acuan yang jelas dan terukur dalam proses pengembangan, pengujian, serta evaluasi proyek Sistem Database Terdistribusi pada mata kuliah Pemrosesan Data Terdistribusi.